Neural Networks

Hrachya Asatryan

Neural Networks

What is the intuition behind the setup?

Neural Networks(NN) were created a a way to mimic how the brain works. They were widely used in the 80s and 90s, and their popularity diminished overtime, until the recent boom.
Back then we didn't have enough computational power to train **NN**, however nowadays, thanks to the technology boom and the optimizations of many **State-of-the-art** techniques, **NN** are primarily used in **ML**.

Of course Artificial **NN** are not as intricate and complex as the human brain, but they do a good job of mimicking it.

Neural Networks

INFORMATION CIRCA 2012	Computer	Human Brain
Computation Units	10-core Xeon: 10^9 Gates	10^{11} Neurons
Storage Units	10^9 bits RAM, 10^{12} bits disk	10^{11} neurons, 10^{14} synapses
Cycle time	10^{-9} sec	10^{-3} sec
Bandwidth	10^9 bits/sec	10^{14} bits/sec

- Computers are way faster than neurons…
- But there are a lot more neurons than we can reasonably model in modern digital computers, and they all fire in parallel
- Neural networks are designed to be massively parallel
- The brain is effectively a billion times faster

Linear Regression as a Neural Network

In Linear Regression, we try to model the target y by a linear function of the input features x_i:

$$\hat{y} = w_1 x_1 + \ldots + w_d x_d + b$$

Or just

$$\hat{y} = \mathbf{w}^\top \mathbf{x} + b$$

for a single sample. For the whole dataset:

$$\hat{\mathbf{y}} = \mathbf{X}\mathbf{w} + b$$

Linear Regression as a Neural Network

The goal then is to minimize the error between y^ and y. To do so, we need to now choose the loss function. For instance, we previously used **Least Squares:**

$$L(\mathbf{w}, b) = \frac{1}{n}\sum_{i=1}^{n} l^{(i)}(\mathbf{w}, b) = \frac{1}{n}\sum_{i=1}^{n} \frac{1}{2}\left(\mathbf{w}^\top \mathbf{x}^{(i)} + b - y^{(i)}\right)^2.$$

$$\mathbf{w}^*, b^* = \operatorname*{argmin}_{\mathbf{w}, b} L(\mathbf{w}, b).$$

Why? It had an analytic solution:

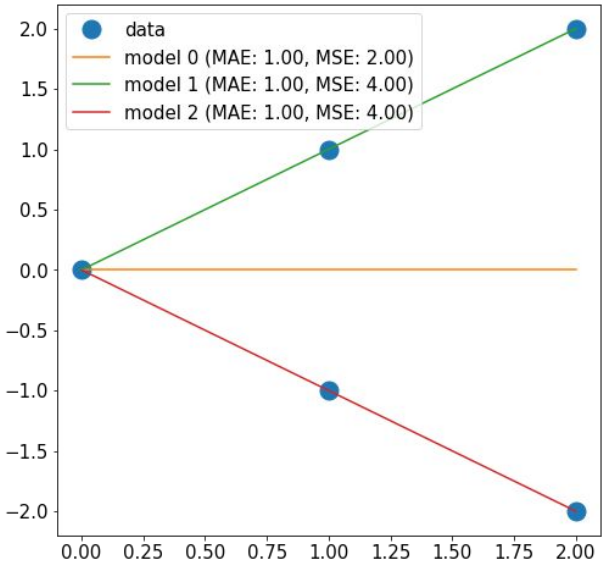
$$\frac{\partial L(\mathbf{w})}{\partial \mathbf{w}} = \mathbf{X}^\top \mathbf{X}\mathbf{w} + \mathbf{X}^\top \mathbf{X}\mathbf{w} - 2\mathbf{X}^\top y = 0$$

$$\mathbf{X}^\top \mathbf{X}\mathbf{w} = \mathbf{X}^\top y$$

$$\mathbf{w}^* = (\mathbf{X}^\top \mathbf{X})^{-1}\mathbf{X}^\top \mathbf{y}.$$

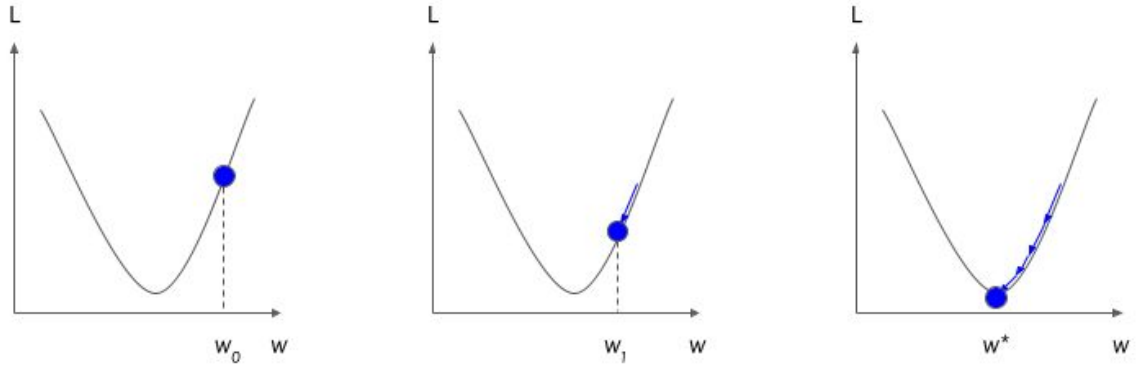
Linear Regression as a Neural Network

The loss function is important. We could also use an **Absolute Distance** function as a loss function:



Linear Regression as a Neural Network

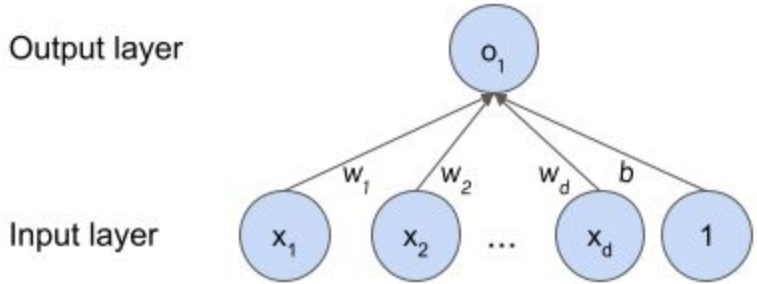
We then **optimized our loss function** with respect to **our weights:**



For this we can use Gradient Descent, or other optimization algorithms.

Linear Regression as a Neural Network

Let's model our Linear Regression model as a graph:

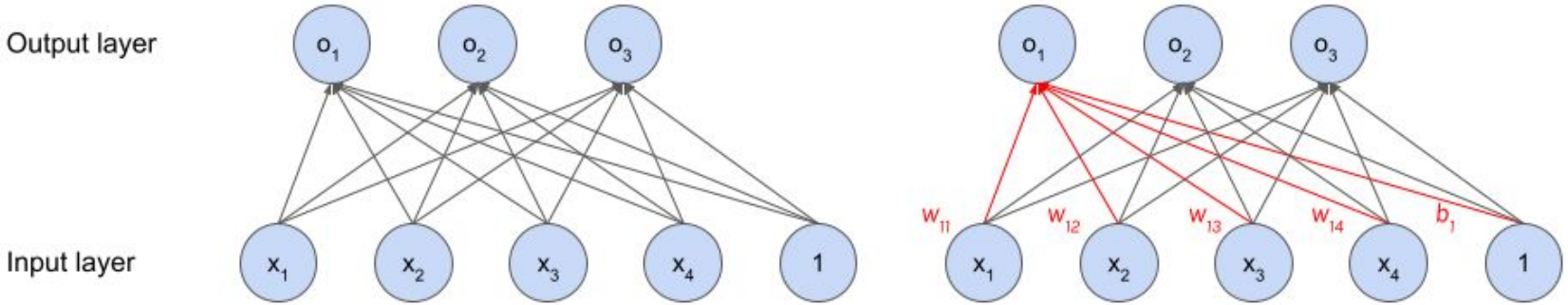


Each node (neuron) in the Output layer is equal to the sum of weight * node for each node (neuron) in the input layer: $o_1 = x_1 w_1 + \ldots + x_d w_d + b = \mathbf{w}^T\mathbf{x} + b$
The output layer here is called fully-connected (or dense), since all neurons from the previous layer are connected to all neurons from that layer.

Classification as a Neural Network

Let's try to do the same for a classification problem. Let's say we are trying to predict what fruit is pictured from a 2x2 image (4 pixels). We can generally model such a classification problem by independently calculating a 'score' associated with each of the three class as follows:

$$o_1 = x_1 w_{11} + x_2 w_{12} + x_3 w_{13} + x_4 w_{14} + b_1,$$
$$o_2 = x_1 w_{21} + x_2 w_{22} + x_3 w_{23} + x_4 w_{24} + b_2,$$
$$o_3 = x_1 w_{31} + x_2 w_{32} + x_3 w_{33} + x_4 w_{34} + b_3.$$



Classification as a Neural Network

To move from abstract ‘scores’ to probabilities, we can ‘activate’ the output layer (which is fully connected again) by a **Softmax function.**

$$\hat{\mathbf{y}} = \mathrm{softmax}(\mathbf{o}) \quad \text{where} \quad \hat{y}_j = \frac{\exp(o_j)}{\sum_k \exp(o_k)}.$$

o_i are typically called **logits.** Note that softmax neither changes the order, nor the relative magnitudes of the inputs:

$$\operatorname*{argmax}_j \hat{y}_j = \operatorname*{argmax}_j o_j.$$

Classification as a Neural Network

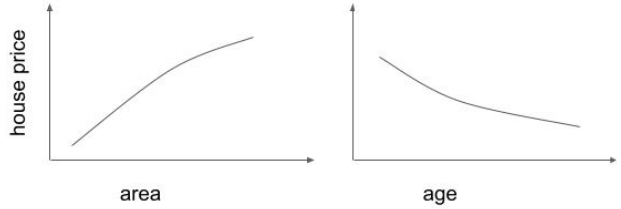
We must once again choose a loss function. Here we are going to choose a loss function that **maximizes the likelihood** of the observations, or equivalently minimizes the negative log likelihood. This is called the **Cross-Entropy Loss**:

$$l(\mathbf{y}, \hat{\mathbf{y}}) = -\sum_{j=1}^{q} y_j \log \hat{y}_j.$$

Summed over all data points.

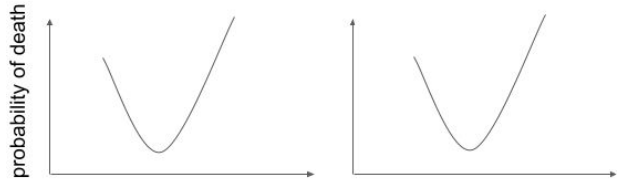
Hidden Layers

Linear Models May Go Wrong



monotonic ✓

linear ✓ (approximately)

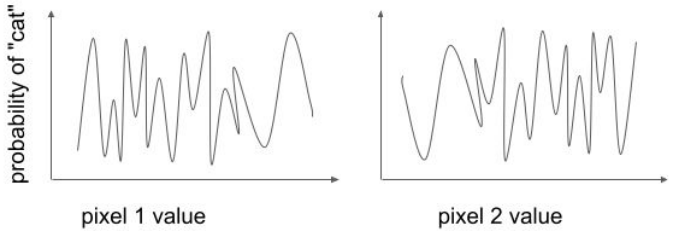


monotonic ✘

linear ✘

Feature engineering:

- $\Delta T = abs(T - T_{normal})$
- $\Delta P = abs(P - P_{normal})$



monotonic ✘

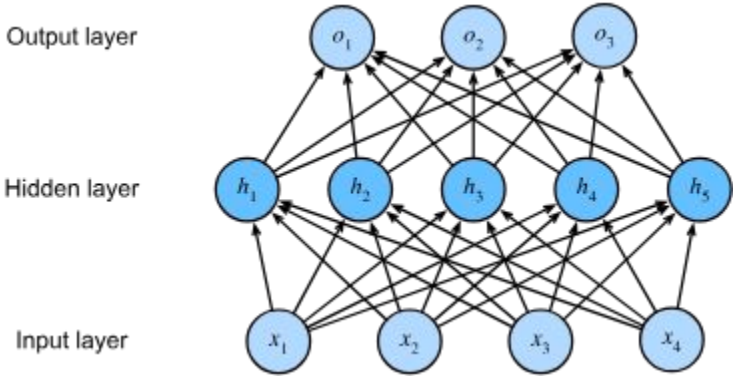
linear ✘

Feature engineering:

- ?

Hidden Layers

We can overcome these limitations of linear models by incorporating one or more **hidden layers**. The easiest way to do this is to stack many fully-connected layers on top of each other. Each layer feeds into the layer above it, until we generate outputs. This architecture is commonly called a **multilayer perceptron**, often abbreviated as **MLP**:



Hidden Layers

Does this even help?

$$\mathbf{H} = \mathbf{X}\mathbf{W}^{(1)} + \mathbf{b}^{(1)},$$
$$\mathbf{O} = \mathbf{H}\mathbf{W}^{(2)} + \mathbf{b}^{(2)}.$$

Plugging in **H** to the second equation:

$$\mathbf{O} = (\mathbf{X}\mathbf{W}^{(1)} + \mathbf{b}^{(1)})\mathbf{W}^{(2)} + \mathbf{b}^{(2)} = \mathbf{X}\mathbf{W}^{(1)}\mathbf{W}^{(2)} + \mathbf{b}^{(1)}\mathbf{W}^{(2)} + \mathbf{b}^{(2)} = \mathbf{X}\mathbf{W} + \mathbf{b}.$$

Our model is still linear with different weights!

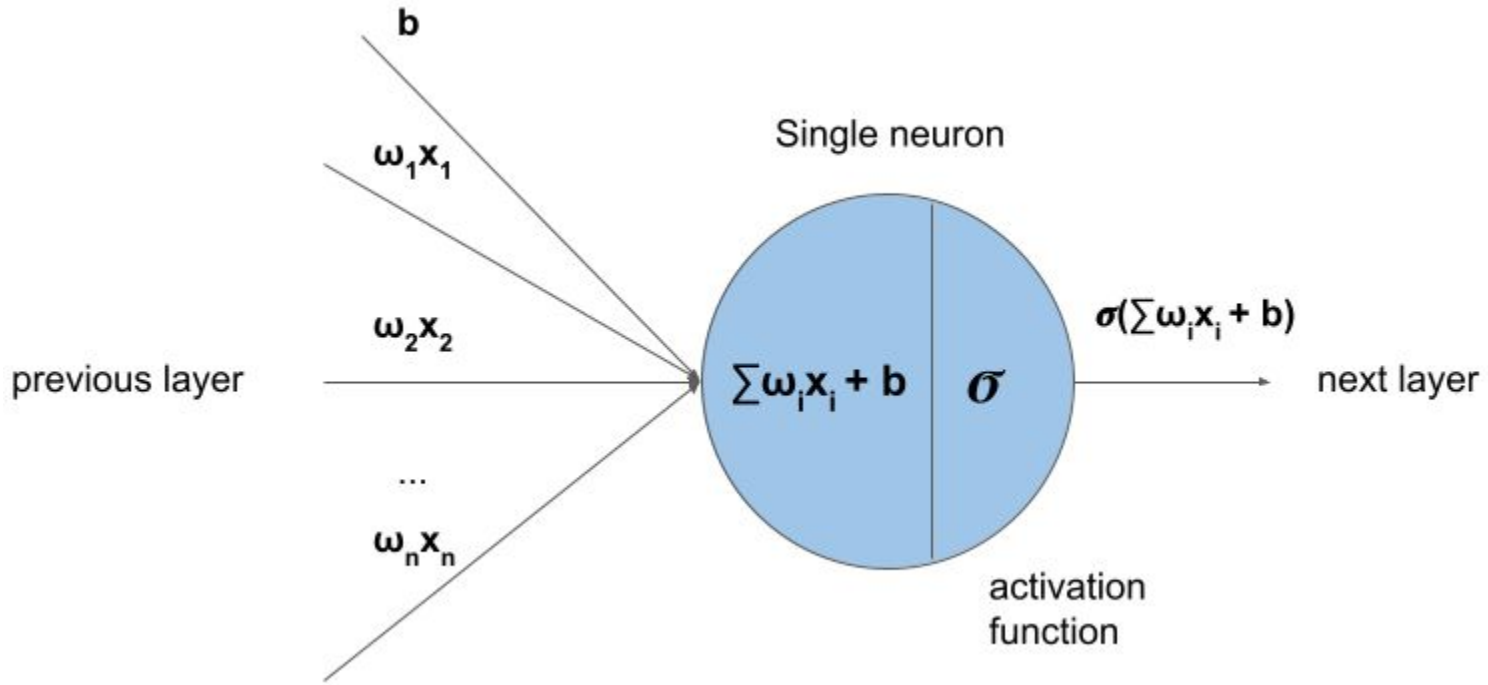
Hidden Layers

Activation Functions are the solution

In order to realize the potential of multilayer architectures, we need one more key ingredient: **a nonlinear activation function σ** to be applied to each hidden unit. In general, with activation functions in place, it is no longer possible to collapse our MLP into a linear model:

$$\mathbf{H} = \sigma(\mathbf{X}\mathbf{W}^{(1)} + \mathbf{b}^{(1)}),$$
$$\mathbf{O} = \mathbf{H}\mathbf{W}^{(2)} + \mathbf{b}^{(2)}.$$

Illustration



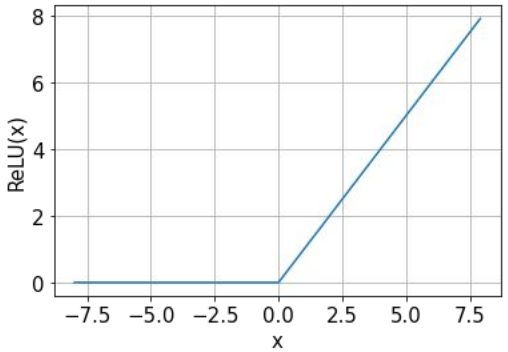
Activation functions

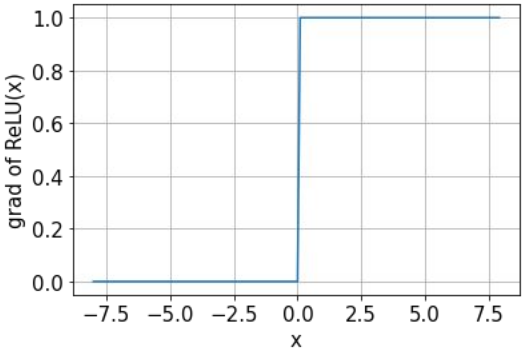
What can we use as activation functions?

Activation functions need to be **differentiable** for us to be able to use Gradient Descent for optimization. They also need to be non-linear. Common choices are:

ReLU

$$\mathrm{ReLU}(x) = \max(x, 0).$$

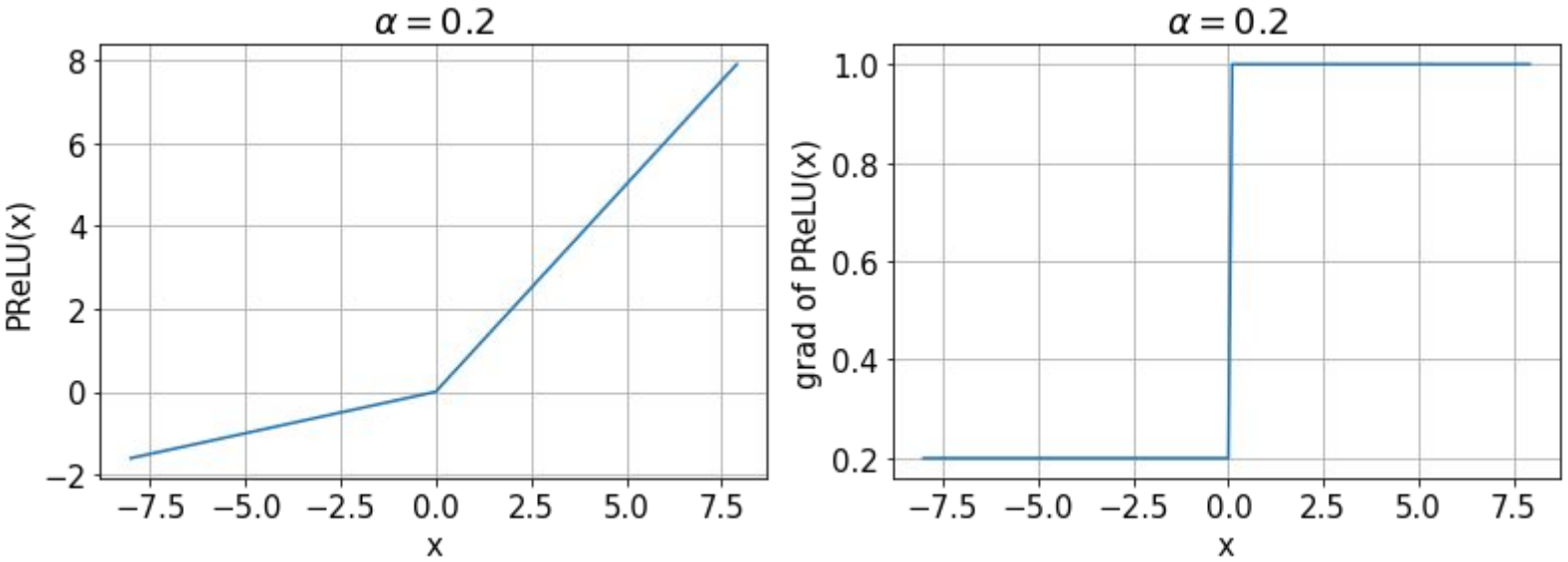




Activation functions

PReLU:

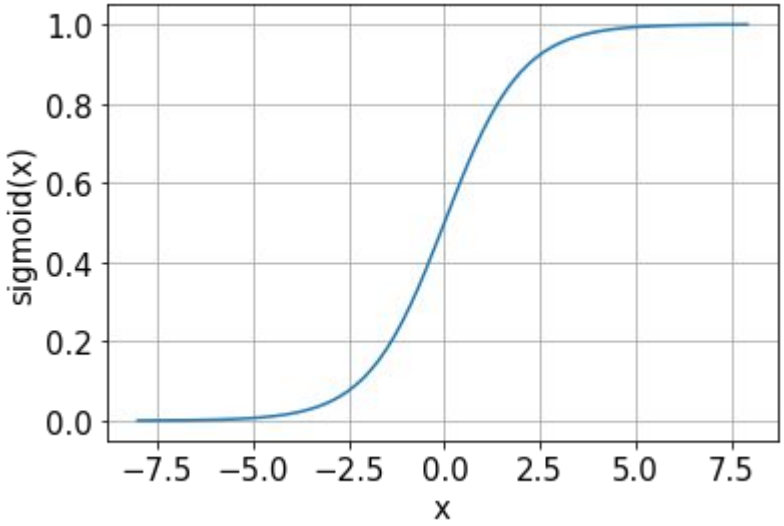
$$\mathrm{pReLU}(x) = \max(0, x) + \alpha \min(0, x).$$

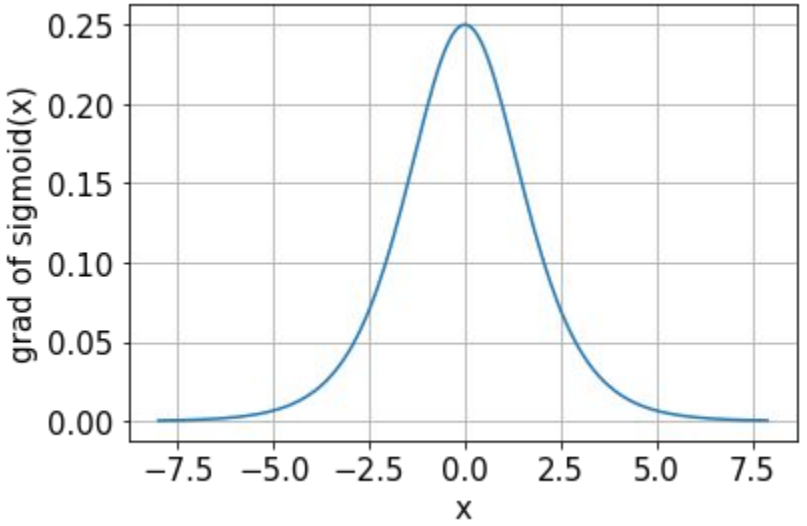


Activation functions

Sigmoid:

$$\text{sigmoid}(x) = \frac{1}{1 + \exp(-x)}.$$

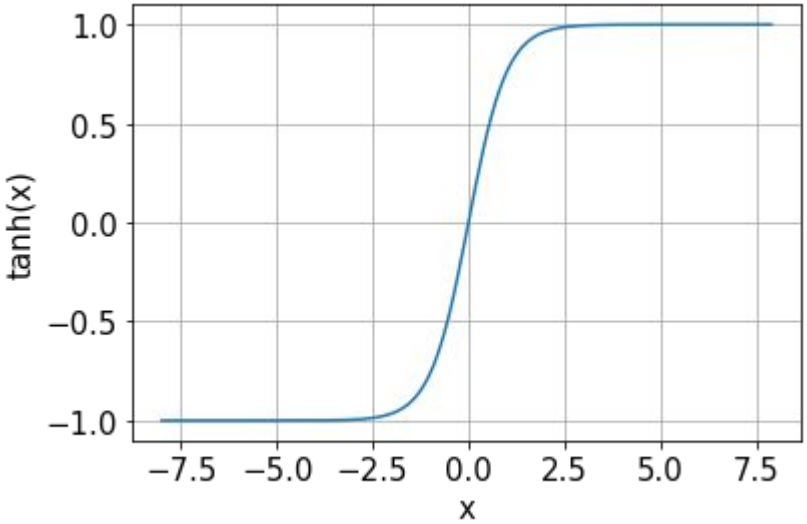


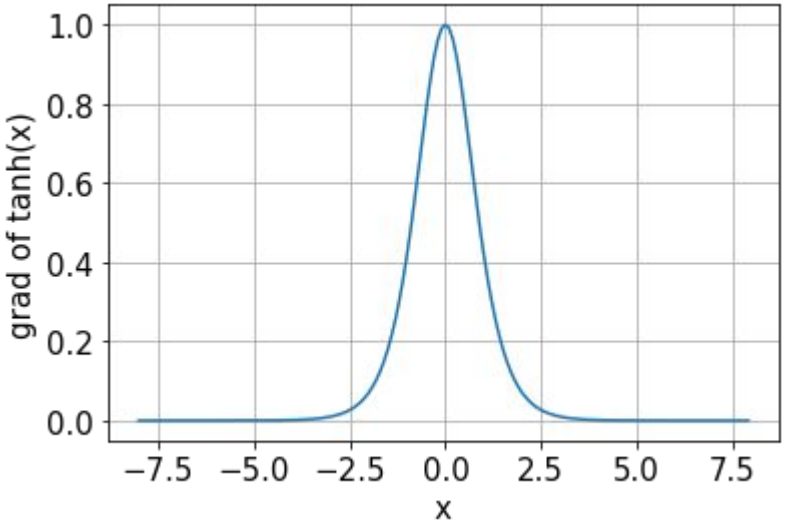


Activation functions

Tanh:

$$\tanh(x) = \frac{1 - \exp(-2x)}{1 + \exp(-2x)}.$$

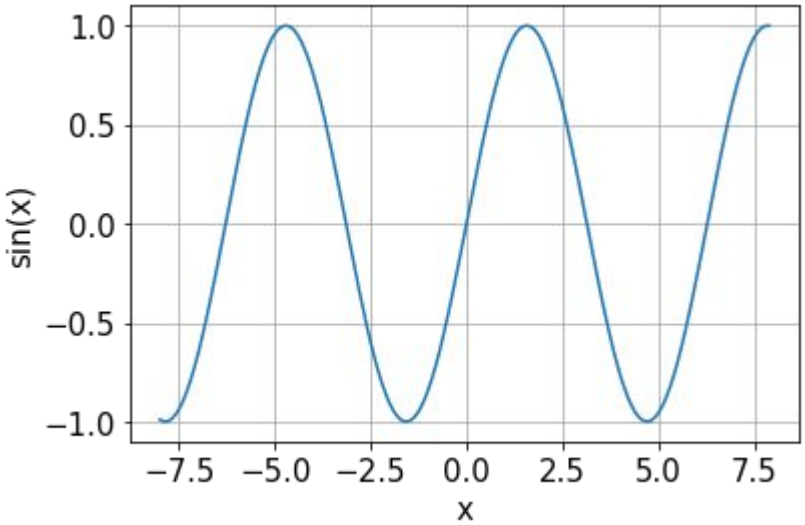


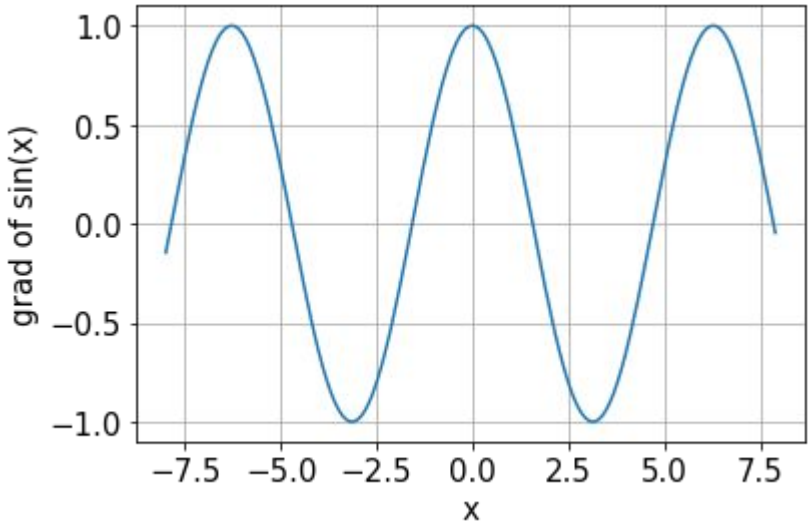


$$\tanh(x) = 2\,\mathrm{sigmoid}(2x) - 1$$

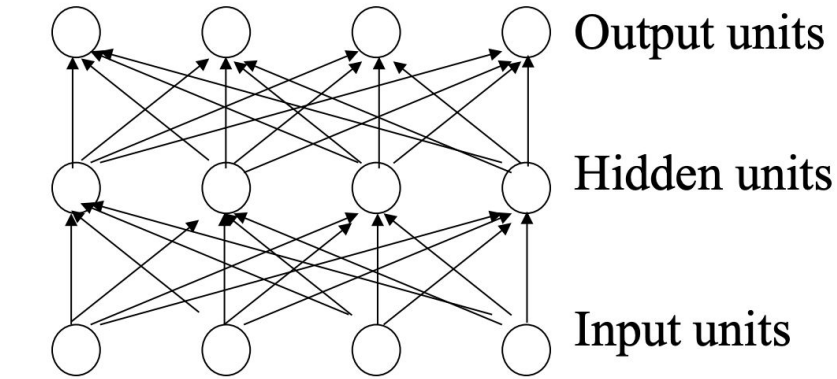
Activation functions

Sin function: A few 2020 studies showed that sinusoidal (periodic) activation functions can be very useful for low dimensional representations, especially when the y(x) has high frequency components:





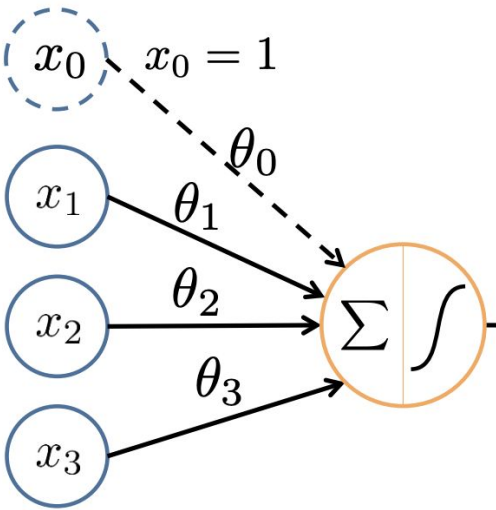
Neural Networks



Layered feed-forward network

- Neural networks are made up of **nodes** or **units**, connected by **links**
- Each link has an associated **weight** and **activation level**
- Each node has an **input function** (typically summing over weighted inputs), an **activation function**, and an **output**

Neuron Model: Logistic Unit

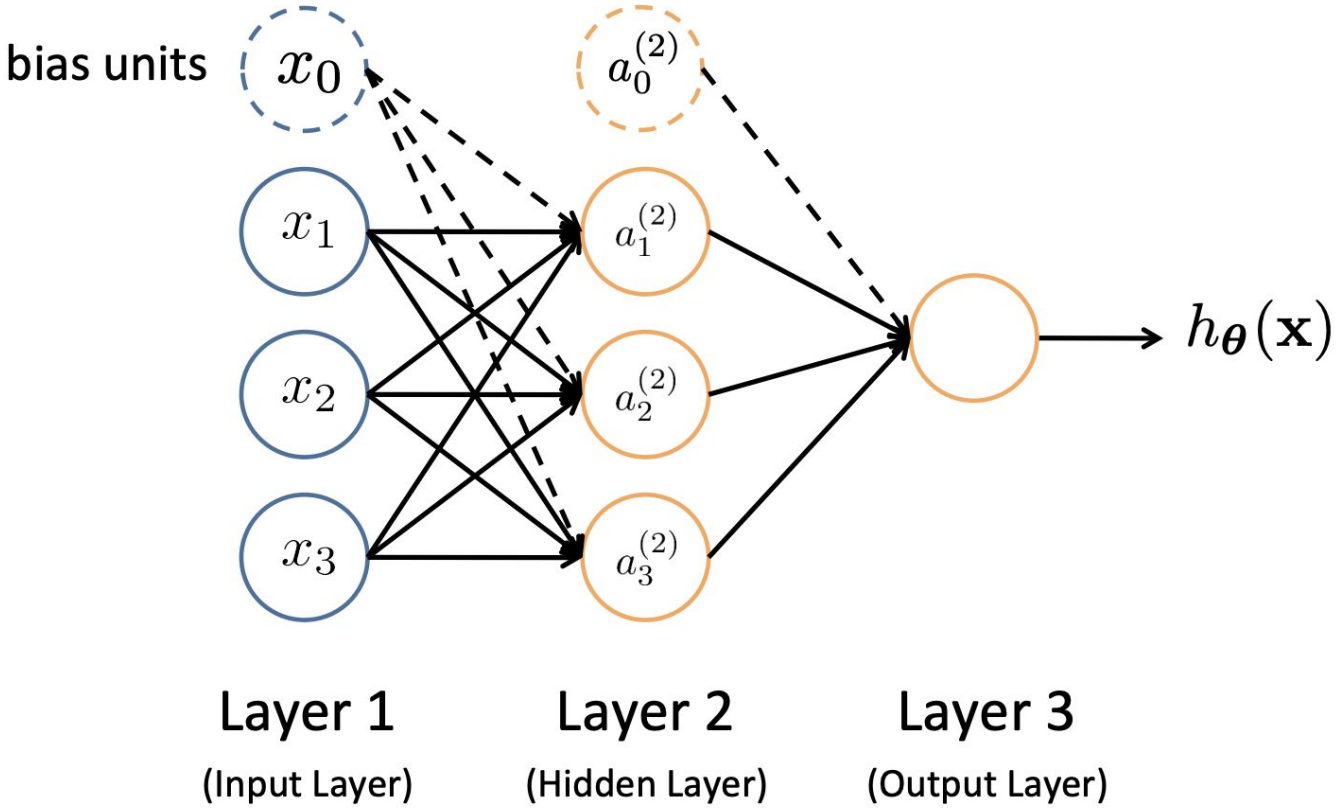


$$\mathbf{x} = \begin{bmatrix} x_0 \\ x_1 \\ x_2 \\ x_3 \end{bmatrix} \qquad \boldsymbol{\theta} = \begin{bmatrix} \theta_0 \\ \theta_1 \\ \theta_2 \\ \theta_3 \end{bmatrix}$$

$$h_{\boldsymbol{\theta}}(\mathbf{x}) = g\left(\boldsymbol{\theta}^{\mathsf{T}}\mathbf{x}\right) = \frac{1}{1 + e^{-\boldsymbol{\theta}^{\mathsf{T}}\mathbf{x}}}$$

Sigmoid (logistic) activation function: $g(z) = \dfrac{1}{1 + e^{-z}}$

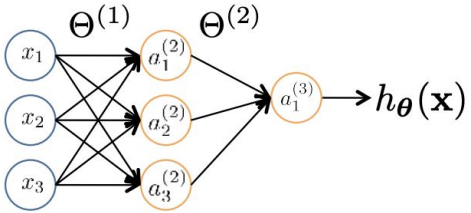
Neuron Model: Logistic Unit



Forward Pass

- **Input Layer**
  Units in the input layer are set by some external function (think of these as sensors), which causes their output links to be activated at the specified level.
- **Forward Propagation**
  Working forward through the network:
  - **Input Function**
    For each unit, the input function is applied to compute the input value. – Usually, this is just the weighted sum of the activations on the links feeding into the node.
  - **Activation Function**
    The activation function transforms this input value into a final output. – Typically, this is a nonlinear function, often a sigmoid function, corresponding to the “threshold” behavior of that node.

Forward Pass



$a_i^{(j)}$ = "activation" of unit i in layer j

$\Theta^{(j)}$ = weight matrix controlling function mapping from layer j to layer $j+1$

$$a_1^{(2)} = g(\Theta_{10}^{(1)} x_0 + \Theta_{11}^{(1)} x_1 + \Theta_{12}^{(1)} x_2 + \Theta_{13}^{(1)} x_3)$$
$$a_2^{(2)} = g(\Theta_{20}^{(1)} x_0 + \Theta_{21}^{(1)} x_1 + \Theta_{22}^{(1)} x_2 + \Theta_{23}^{(1)} x_3)$$
$$a_3^{(2)} = g(\Theta_{30}^{(1)} x_0 + \Theta_{31}^{(1)} x_1 + \Theta_{32}^{(1)} x_2 + \Theta_{33}^{(1)} x_3)$$
$$h_\Theta(x) = a_1^{(3)} = g(\Theta_{10}^{(2)} a_0^{(2)} + \Theta_{11}^{(2)} a_1^{(2)} + \Theta_{12}^{(2)} a_2^{(2)} + \Theta_{13}^{(2)} a_3^{(2)})$$

If network has s_j units in layer j *and* s_{j+1} units in layer $j+1$, then $\Theta^{(j)}$ has dimension $s_{j+1} \times (s_j+1)$.

$$\Theta^{(1)} \in \mathbb{R}^{3\times 4} \qquad \Theta^{(2)} \in \mathbb{R}^{1\times 4}$$

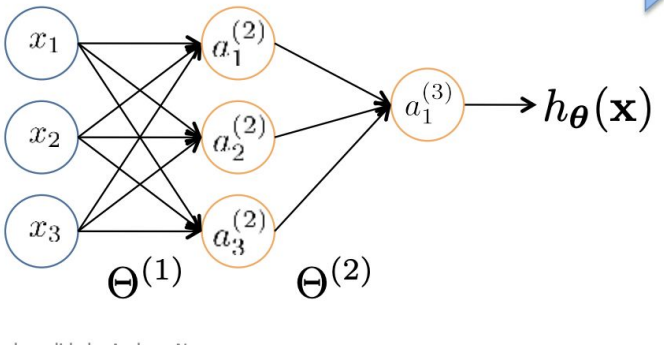
Forward Pass

$$a_1^{(2)} = g\left(\Theta_{10}^{(1)}x_0 + \Theta_{11}^{(1)}x_1 + \Theta_{12}^{(1)}x_2 + \Theta_{13}^{(1)}x_3\right) = g\left(z_1^{(2)}\right)$$

$$a_2^{(2)} = g\left(\Theta_{20}^{(1)}x_0 + \Theta_{21}^{(1)}x_1 + \Theta_{22}^{(1)}x_2 + \Theta_{23}^{(1)}x_3\right) = g\left(z_2^{(2)}\right)$$

$$a_3^{(2)} = g\left(\Theta_{30}^{(1)}x_0 + \Theta_{31}^{(1)}x_1 + \Theta_{32}^{(1)}x_2 + \Theta_{33}^{(1)}x_3\right) = g\left(z_3^{(2)}\right)$$

$$h_\Theta(\mathbf{x}) = g\left(\Theta_{10}^{(2)}a_0^{(2)} + \Theta_{11}^{(2)}a_1^{(2)} + \Theta_{12}^{(2)}a_2^{(2)} + \Theta_{13}^{(2)}a_3^{(2)}\right) = g\left(z_1^{(3)}\right)$$



Feed-Forward Steps:

$$\mathbf{z}^{(2)} = \Theta^{(1)}\mathbf{x}$$

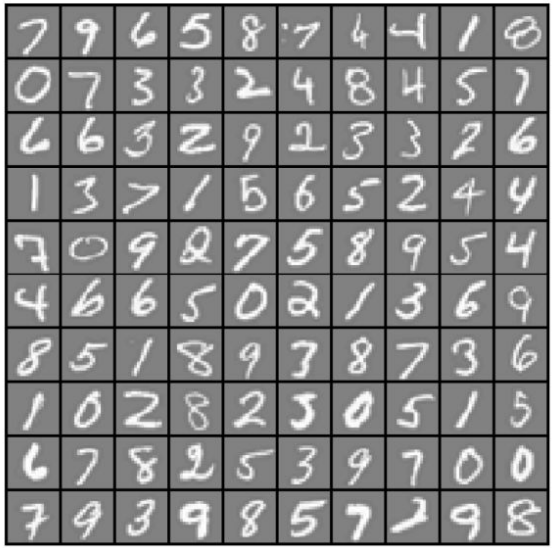
$$\mathbf{a}^{(2)} = g(\mathbf{z}^{(2)})$$

Add $a_0^{(2)} = 1$

$$\mathbf{z}^{(3)} = \Theta^{(2)}\mathbf{a}^{(2)}$$

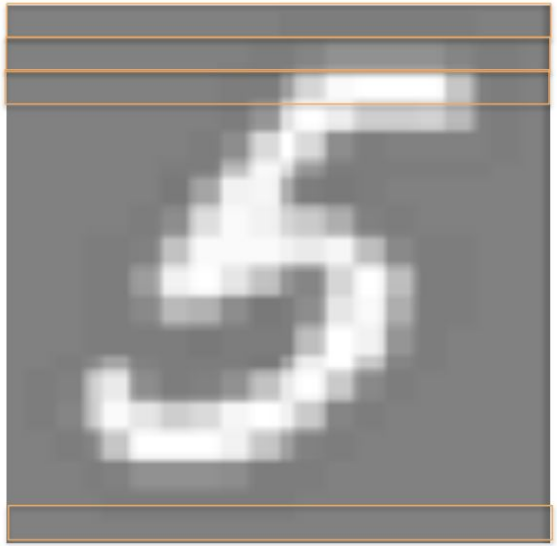
$$h_\Theta(\mathbf{x}) = \mathbf{a}^{(3)} = g(\mathbf{z}^{(3)})$$

Layering Representation



20 × 20 pixel images

d = 400 10 classes



$x_1 \dots x_{20}$

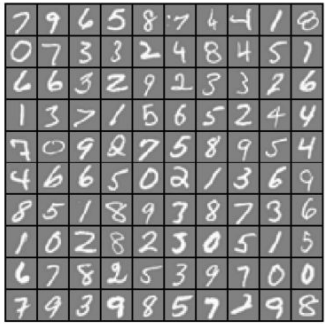
$x_{21} \dots x_{40}$

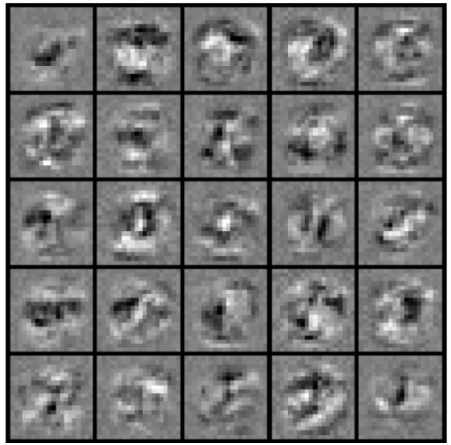
$x_{41} \dots x_{60}$

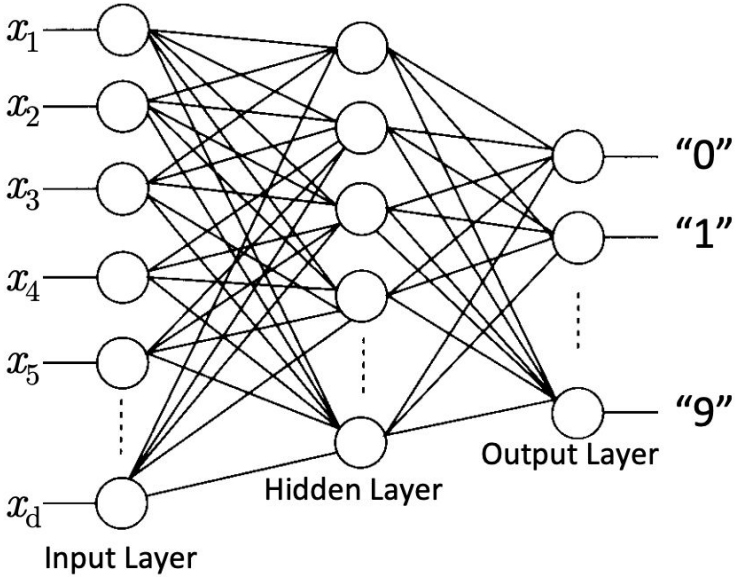
$x_{381} \dots x_{400}$

Each image is “unrolled” into a vector $\mathbf{x}$ of pixel intensities

Layering Representation







Visualization of Hidden Layer

Perceptron Learning Rule

Main Algorithm:

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \alpha(y - h(\mathbf{x}))\mathbf{x}$$

Equivalent to the intuitive rules:

- If the output is correct, don't change the weights
- If the output is low *(h(x) = 0, y = 1)*, increment the weights for all the inputs which are 1.
- If the output is high *(h(x) = 1, y = 0)*, decrement the weights for all the inputs which are 1.

Perceptron Convergence Theorem:

If there is a set of weights that is consistent with the training data (i.e., the data is linearly separable), the perceptron learning algorithm will converge.

In fact, **MLPs are universal approximators**: MLPs with as few as one hidden layer using arbitrary squashing functions are capable of approximating any Borel measurable function from one finite dimensional space to another to any desired degree of accuracy, provided sufficiently many hidden units are available.

Perceptron Learning Rule

Given training data $\{(\boldsymbol{x}^{(i)}, y^{(i)})\}_{i=1}^{n}$
Let $\boldsymbol{\theta} \leftarrow [0, 0, \ldots, 0]$
Repeat:
    Let $\boldsymbol{\Delta} \leftarrow [0, 0, \ldots, 0]$
    for $i = 1 \ldots n$, do
        if $y^{(i)}\boldsymbol{x}^{(i)}\boldsymbol{\theta} \leq 0$ // prediction for ith instance is incorrect
            $\boldsymbol{\Delta} \leftarrow \boldsymbol{\Delta} + y^{(i)}\boldsymbol{x}^{(i)}$
    $\boldsymbol{\Delta} \leftarrow \boldsymbol{\Delta}/n$ // compute average update
    $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \alpha\boldsymbol{\Delta}$
Until $\|\boldsymbol{\Delta}\|_2 < \epsilon$

Simplest case: If α = 1 and we don't normalize, this yields the fixed increment perceptron.
Each increment of the outer loop is called an epoch.

Learning in Neural Networks

Main Algorithm:

Similar to the perceptron learning algorithm, we cycle through our examples:

- If the outputs of the neural networks are correct, no changes are made.
- If there is an error, weights are adjusted to reduce the error.

The trick is to assess the blame for the error and divide it among contributing weights.
In case of Logistic Regression, we had Cross Entropy given as:

$$J(\theta) = -\frac{1}{n}\sum_{i=1}^{n}[y_i \log h_{\boldsymbol{\theta}}(\mathbf{x}_i) + (1-y_i)\log\left(1-h_{\boldsymbol{\theta}}(\mathbf{x}_i)\right)] + \frac{\lambda}{2n}\sum_{j=1}^{d}\theta_j^2$$

In this case, we give our cost function as:

$$h_\Theta \in \mathbb{R}^K \qquad (h_\Theta(\mathbf{x}))_i = i^{th}\text{output}$$

$$J(\Theta) = -\frac{1}{n}\left[\sum_{i=1}^{n}\sum_{k=1}^{K} y_{ik}\log\left(h_\Theta(\mathbf{x}_i)\right)_k + (1-y_{ik})\log\left(1-(h_\Theta(\mathbf{x}_i))_k\right)\right]$$

$$+\frac{\lambda}{2n}\sum_{l=1}^{L-1}\sum_{i=1}^{s_{l-1}}\sum_{j=1}^{s_l}\left(\Theta_{ji}^{(l)}\right)^2$$

k^{th} class: true, predicted
not k^{th} class: true, predicted

Optimizing the Neural Net

$$J(\Theta) = -\frac{1}{n}\left[\sum_{i=1}^{n}\sum_{k=1}^{K} y_{ik}\log(h_\Theta(\mathbf{x}_i))_k + (1-y_{ik})\log\Big(1-(h_\Theta(\mathbf{x}_i))_k\Big)\right]$$
$$+\frac{\lambda}{2n}\sum_{l=1}^{L-1}\sum_{i=1}^{s_{l-1}}\sum_{j=1}^{s_l}\left(\Theta_{ji}^{(l)}\right)^2$$

Solve via: $\min_\Theta J(\Theta)$

$J(\Theta)$ is not convex, so GD on a neural net yields a local optimum

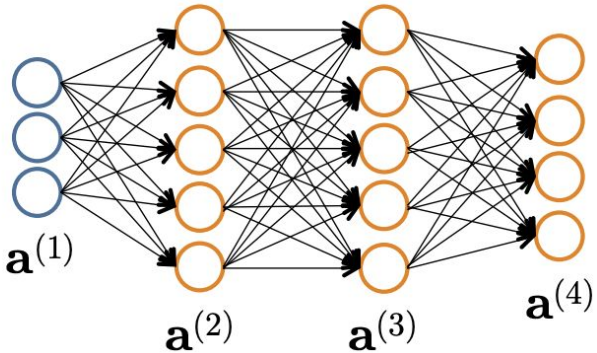
- But, tends to work well in practice

Need code to compute:

- $J(\Theta)$
- $\frac{\partial}{\partial\Theta_{ij}^{(l)}}J(\Theta)$

Forward Propagation

- Given one labeled training instance $(\mathbf{x}, y)$:



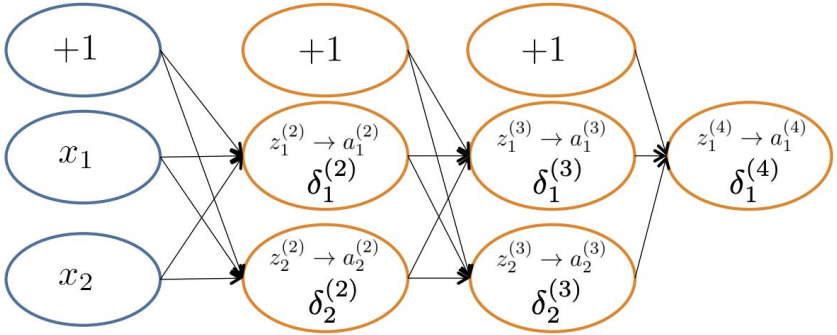
Forward Propagation

- $\mathbf{a}^{(1)} = \mathbf{x}$
- $\mathbf{z}^{(2)} = \Theta^{(1)}\mathbf{a}^{(1)}$
- $\mathbf{a}^{(2)} = g(\mathbf{z}^{(2)})$ [add $a_0^{(2)}$]
- $\mathbf{z}^{(3)} = \Theta^{(2)}\mathbf{a}^{(2)}$
- $\mathbf{a}^{(3)} = g(\mathbf{z}^{(3)})$ [add $a_0^{(3)}$]
- $\mathbf{z}^{(4)} = \Theta^{(3)}\mathbf{a}^{(3)}$
- $\mathbf{a}^{(4)} = h_\Theta(\mathbf{x}) = g(\mathbf{z}^{(4)})$

Back Propagation

Main Algorithm:

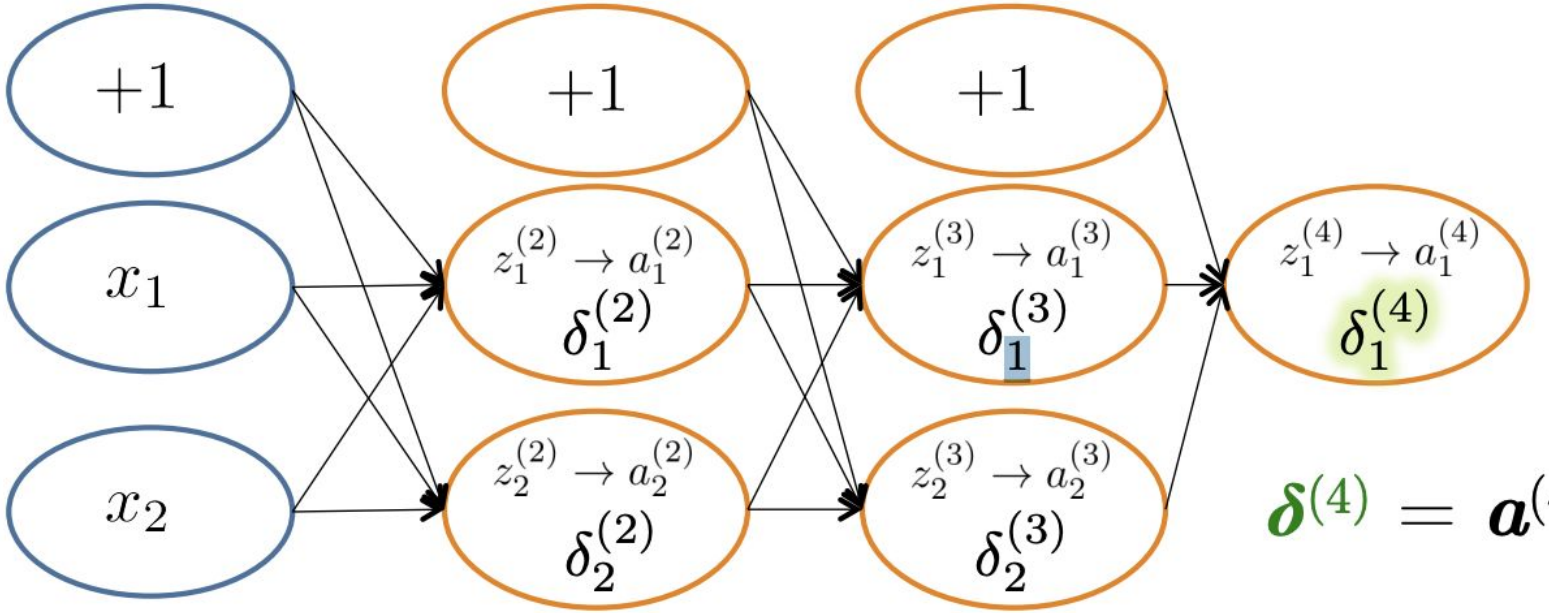
Each hidden node ***j*** is "responsible" for some fraction of the error $\delta_j^{(l)}$ in each of the output nodes to which it connects. $\delta_j^{(l)}$ is divided according to the strength of the connection between the hidden node and the output node. Then the "blame" is propagated back to provide the error values for the hidden layer.



- $\delta_j^{(l)}$ - the "error"of node *j* in layer *l*
- Formally $\delta_j^{(l)} = \dfrac{\partial}{\partial z_j^{(l)}} \text{cost}(\mathbf{x}_i)$

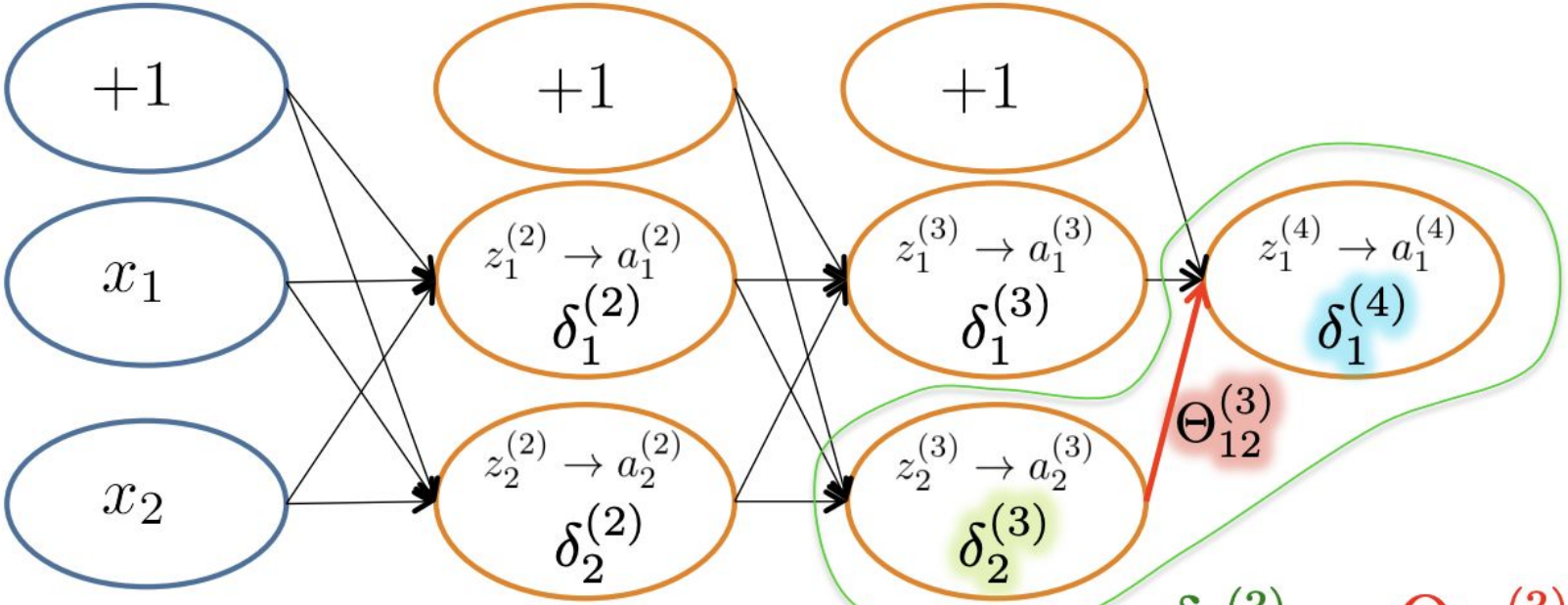
$$\text{cost}(\mathbf{x}_i) = y_i \log h_\Theta(\mathbf{x}_i) + (1 - y_i) \log(1 - h_\Theta(\mathbf{x}_i))$$

Back Propagation



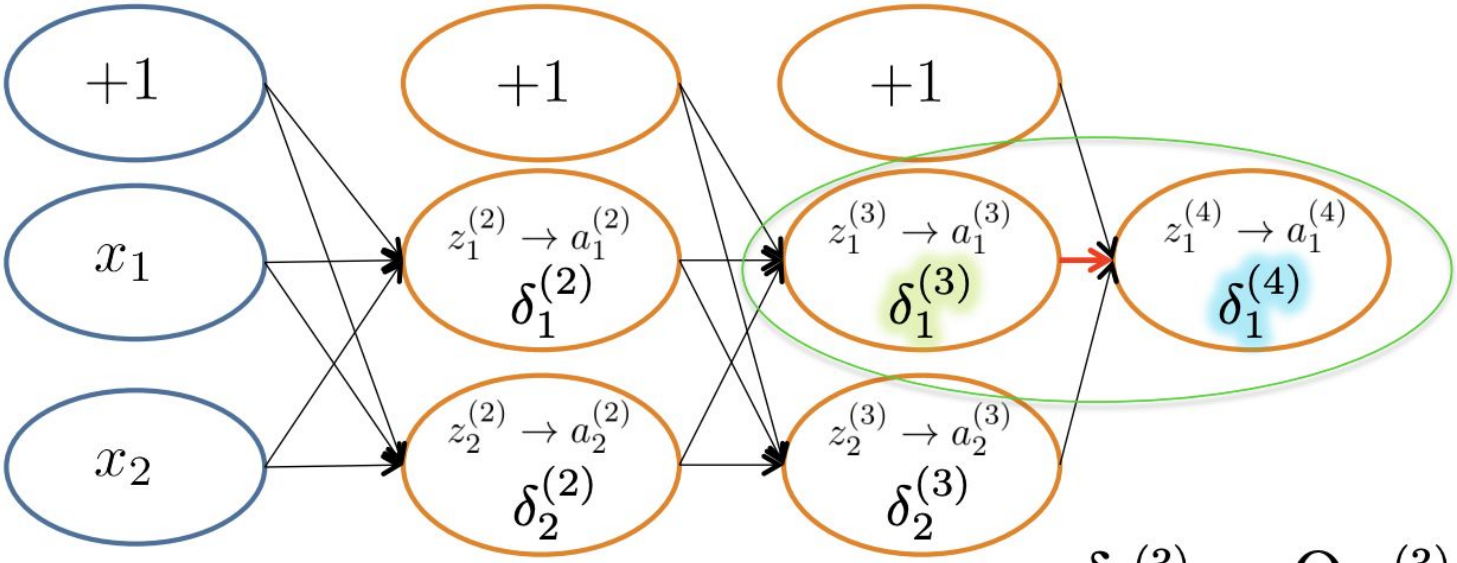
$$\boldsymbol{\delta}^{(4)} = \boldsymbol{a}^{(4)} - \mathbf{y}$$

Back Propagation



$$\delta_2^{(3)} = \Theta_{12}^{(3)} \times \delta_1^{(4)}$$

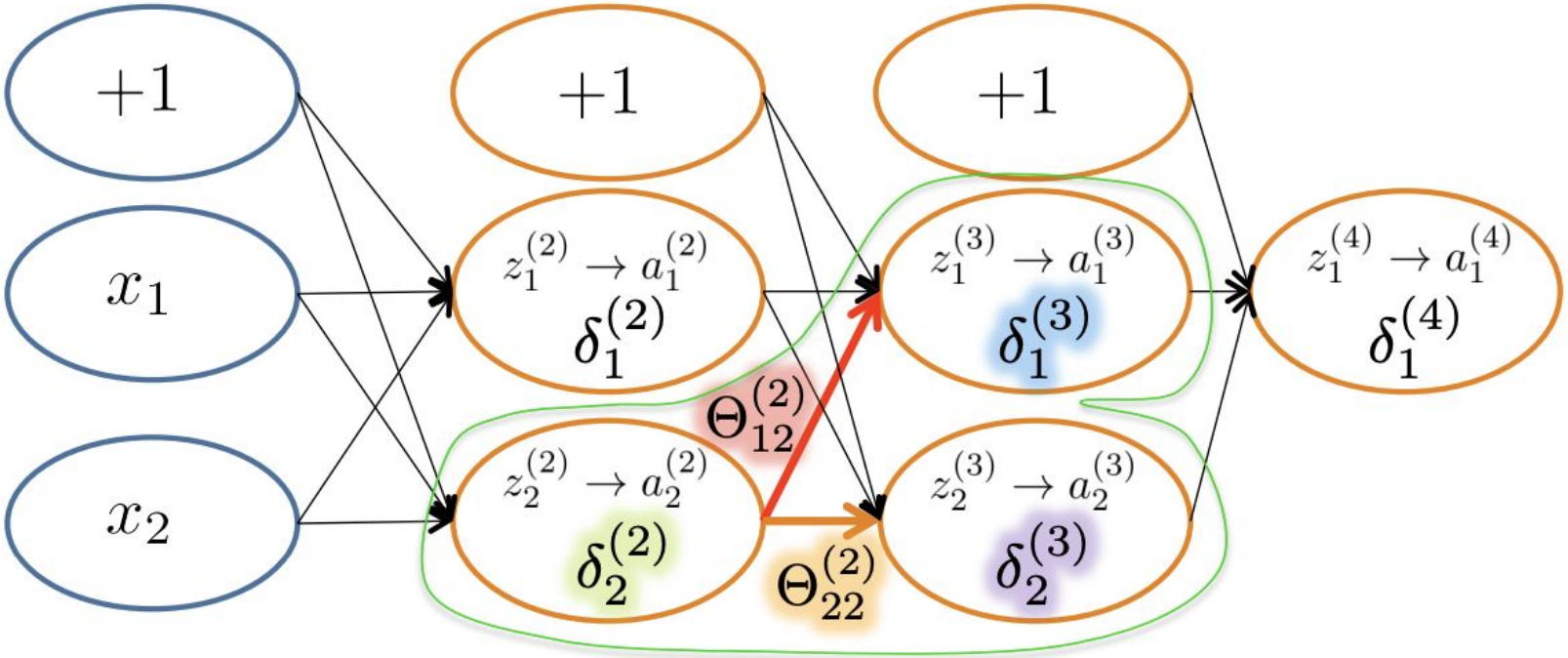
Back Propagation



$$\delta_2^{(3)} = \Theta_{12}^{(3)} \times \delta_1^{(4)}$$

$$\delta_1^{(3)} = \Theta_{11}^{(3)} \times \delta_1^{(4)}$$

Back Propagation

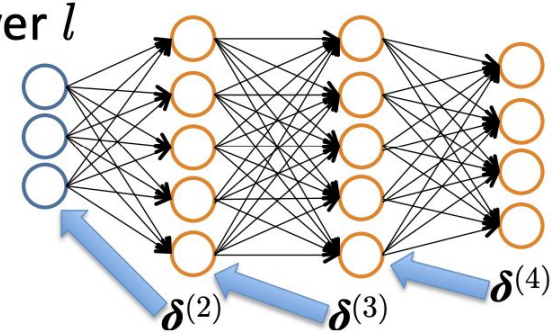


$$\delta_2^{(2)} = \Theta_{12}^{(2)} \times \delta_1^{(3)} + \Theta_{22}^{(2)} \times \delta_2^{(3)}$$

Back Propagation

Let $\delta_j^{(l)}$ = "error" of node j in layer l

(#layers $L = 4$)



Element-wise product .*

Backpropagation

- $\delta^{(4)} = a^{(4)} - \mathbf{y}$
- $\delta^{(3)} = (\Theta^{(3)})^{\mathsf{T}}\delta^{(4)}$.* $g'(\mathbf{z}^{(3)})$
- $\delta^{(2)} = (\Theta^{(2)})^{\mathsf{T}}\delta^{(3)}$.* $g'(\mathbf{z}^{(2)})$
- (No $\delta^{(1)}$)

$g'(\mathbf{z}^{(3)}) = \mathbf{a}^{(3)}$.* $(1-\mathbf{a}^{(3)})$

$g'(\mathbf{z}^{(2)}) = \mathbf{a}^{(2)}$.* $(1-\mathbf{a}^{(2)})$

$$\frac{\partial}{\partial \Theta_{ij}^{(l)}} J(\Theta) = a_j^{(l)} \delta_i^{(l+1)}$$

(ignoring λ; if $\lambda = 0$)

Back Propagation

Set $\Delta_{ij}^{(l)} = 0 \quad \forall l, i, j$ (Used to accumulate gradient)

For each training instance $(\mathbf{x}_i, y_i)$:

Set $\mathbf{a}^{(1)} = \mathbf{x}_i$

Compute $\{\mathbf{a}^{(2)}, \ldots, \mathbf{a}^{(L)}\}$ via forward propagation

Compute $\boldsymbol{\delta}^{(L)} = \mathbf{a}^{(L)} - y_i$

Compute errors $\{\boldsymbol{\delta}^{(L-1)}, \ldots, \boldsymbol{\delta}^{(2)}\}$

Compute gradients $\Delta_{ij}^{(l)} = \Delta_{ij}^{(l)} + a_j^{(l)} \delta_i^{(l+1)}$

Compute avg regularized gradient $D_{ij}^{(l)} = \begin{cases} \frac{1}{n}\Delta_{ij}^{(l)} + \lambda\Theta_{ij}^{(l)} & \text{if } j \neq 0 \\ \frac{1}{n}\Delta_{ij}^{(l)} & \text{otherwise} \end{cases}$

$\boldsymbol{D}^{(l)}$ is the matrix of partial derivatives of $J(\Theta)$

Note: Can vectorize $\Delta_{ij}^{(l)} = \Delta_{ij}^{(l)} + a_j^{(l)} \delta_i^{(l+1)}$ as $\boldsymbol{\Delta}^{(l)} = \boldsymbol{\Delta}^{(l)} + \boldsymbol{\delta}^{(l+1)} \mathbf{a}^{(l)\mathsf{T}}$

Training a Neural Network

Given: training set $\{(\mathbf{x}_1, y_1), \ldots, (\mathbf{x}_n, y_n)\}$
Initialize all $\Theta^{(l)}$ randomly (NOT to 0!)
Loop // each iteration is called an epoch

Set $\Delta_{ij}^{(l)} = 0 \quad \forall l, i, j$ (Used to accumulate gradient)

For each training instance $(\mathbf{x}_i, y_i)$:

Set $\mathbf{a}^{(1)} = \mathbf{x}_i$

Compute $\{\mathbf{a}^{(2)}, \ldots, \mathbf{a}^{(L)}\}$ via forward propagation

Compute $\boldsymbol{\delta}^{(L)} = \mathbf{a}^{(L)} - y_i$

Compute errors $\{\boldsymbol{\delta}^{(L-1)}, \ldots, \boldsymbol{\delta}^{(2)}\}$

Compute gradients $\Delta_{ij}^{(l)} = \Delta_{ij}^{(l)} + a_j^{(l)} \delta_i^{(l+1)}$

Compute avg regularized gradient $D_{ij}^{(l)} = \begin{cases} \frac{1}{n}\Delta_{ij}^{(l)} + \lambda\Theta_{ij}^{(l)} & \text{if } j \neq 0 \\ \frac{1}{n}\Delta_{ij}^{(l)} & \text{otherwise} \end{cases}$

Update weights via gradient step $\Theta_{ij}^{(l)} = \Theta_{ij}^{(l)} - \alpha D_{ij}^{(l)}$

Until weights converge or max #epochs is reached

Backpropagation

Back Propagation

Problems:

- Back propagation is a black box. We don’t know what exactly is being learned and how. We can have some ideas based on layer-wise visualizations, but at the end of the day we have little idea on what’s going on behind the scenes.
- Back propagation tends to the local minima, which is normal given the complexity of our neural networks, but we need to acknowledge that the solutions given are in most cases not the best ones.

Implementation Details

Random Initialization:

- We need to initialize the initial weight matrices randomly, usually coming from a 0 mean standard distribution, otherwise we end up creating symmetry, causing all neurons in a layer to compute the same function and receive the same gradients, which prevents them from learning diverse features.
- We **can't** have uniform weights for the same reason stated above.

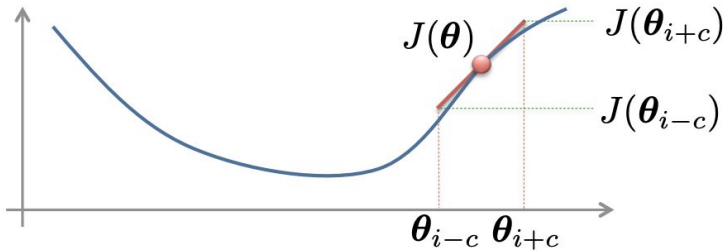
Parameter Tricks:

- For convenience, compress all parameters into $\boldsymbol{\theta}$,
- Each step, check to make sure that $J(\boldsymbol{\theta})$ decreases,
- Implement a gradient--checking procedure to ensure that the gradient is correct.

Implementation Details

Gradient checking:

Idea: estimate the gradient numerically to verify implementation, then turn off gradient checking.



$$\frac{\partial}{\partial \theta_i} J(\boldsymbol{\theta}) \approx \frac{J(\boldsymbol{\theta}_{i+c}) - J(\boldsymbol{\theta}_{i-c})}{2c} \qquad c \approx 1\text{E-4}$$

$$\boldsymbol{\theta}_{i+c} = [\theta_1, \theta_2, \ldots, \theta_{i-1}, \theta_i + c, \theta_{i+1}, \ldots]$$

Change ONLY the i^{th} entry in $\boldsymbol{\theta}$, increasing (or decreasing) it by c

Implementation Details

Gradient checking:

$\boldsymbol{\theta} \in \mathbb{R}^m$ $\boldsymbol{\theta}$ is an "unrolled" version of $\Theta^{(1)}, \Theta^{(2)}, \ldots$

$\boldsymbol{\theta} = [\theta_1, \theta_2, \theta_3, \ldots, \theta_m]$

Put in vector called `gradApprox`

$$\frac{\partial}{\partial \theta_1} J(\boldsymbol{\theta}) \approx \frac{J([\theta_1 + c, \theta_2, \theta_3, \ldots, \theta_m]) - J([\theta_1 - c, \theta_2, \theta_3, \ldots, \theta_m])}{2c}$$

$$\frac{\partial}{\partial \theta_2} J(\boldsymbol{\theta}) \approx \frac{J([\theta_1, \theta_2 + c, \theta_3, \ldots, \theta_m]) - J([\theta_1, \theta_2 - c, \theta_3, \ldots, \theta_m])}{2c}$$

$$\vdots$$

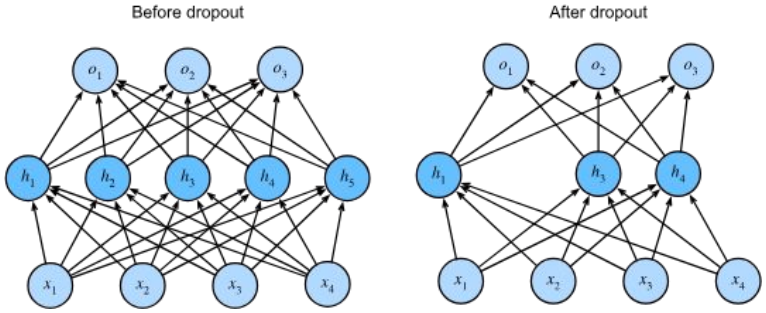
$$\frac{\partial}{\partial \theta_m} J(\boldsymbol{\theta}) \approx \frac{J([\theta_1, \theta_2, \theta_3, \ldots, \theta_m + c]) - J([\theta_1, \theta_2, \theta_3, \ldots, \theta_m - c])}{2c}$$

Dropout

Deep neural networks are capable of overfitting. Training deep nets on **randomly-labeled images**, neural network optimized by stochastic gradient descent could label **every** image in the training set perfectly.
A new type of regularization was proposed: **Dropout.**
In standard dropout regularization, one debiases each layer by normalizing by the fraction of nodes that were retained (not dropped out). In other words, with dropout probability p, each intermediate activation h (output of the neuron) is replaced by a random variable h' as follows:

$$h' = \begin{cases} 0 & \text{with probability } p \\ \frac{h}{1-p} & \text{otherwise} \end{cases}$$

By design, the expectation remains unchanged, i.e., E[h'] = h. Dropout is then disabled at testing or predicting time.



Useful links

- Neural Network Playground
- Why does “Zero Initialization” work?
- 3Blue1Brown Neural Networks Playlist
  - But what is a neural network? | Deep learning chapter 1
  - Gradient descent, how neural networks learn | Deep Learning Chapter 2
  - Backpropagation, intuitively | Deep Learning Chapter 3
  - Backpropagation calculus | Deep Learning Chapter 4
- Dive into Deep Learning
  - 3. Linear Neural Networks for Regression
  - 4. Linear Neural Networks for Classification
  - 5. Multilayer Perceptrons — Dive into Deep Learning 1.0.3 documentation